Homework 6

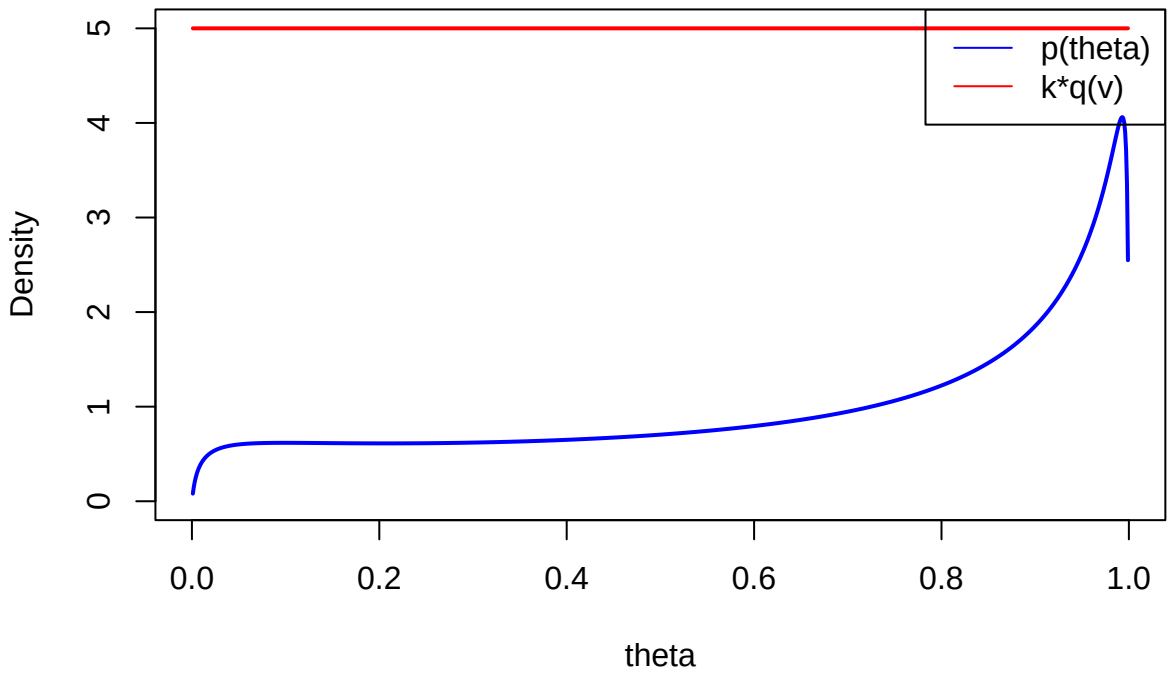
2026-04-02

Jona Nakai DS 4420 Homework #6

Problem 2

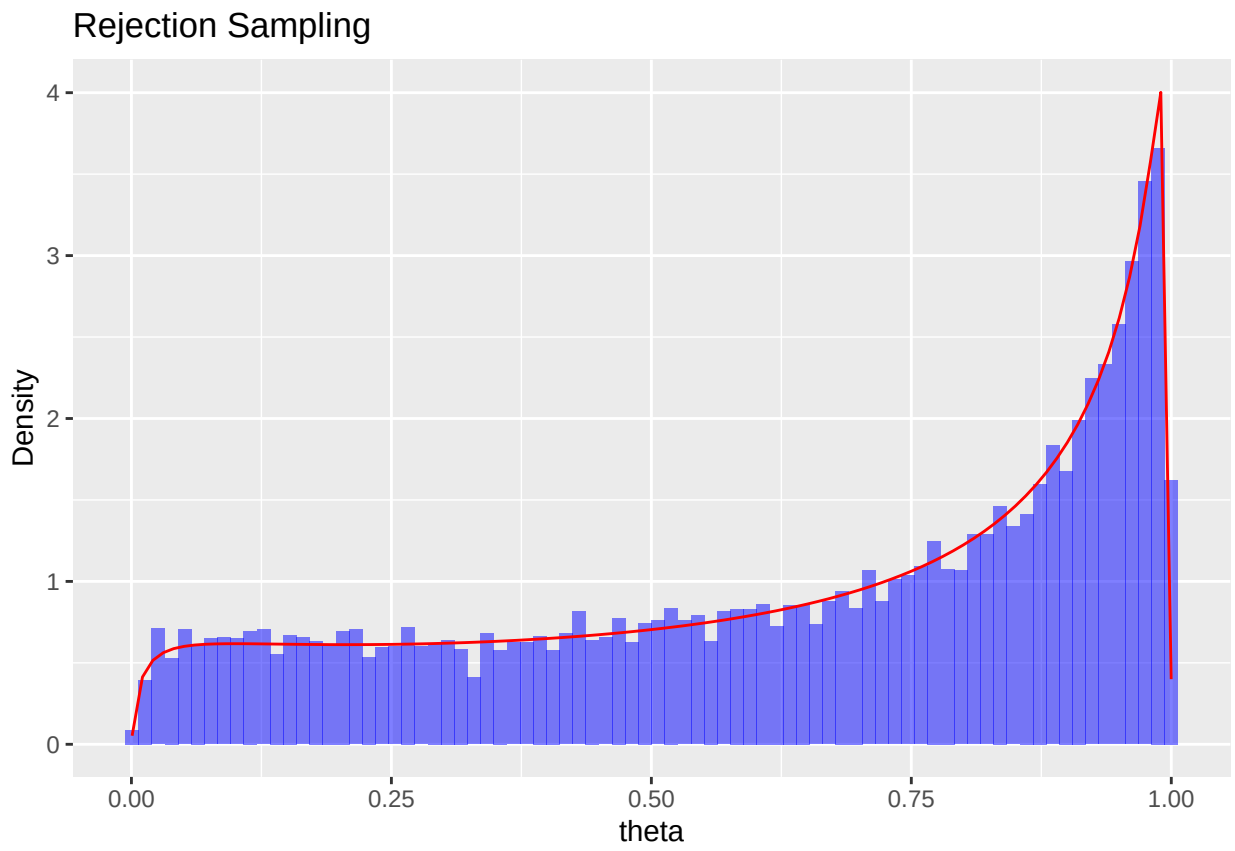
Part (a)

```
set.seed(42)

mu <- 1
sigma <- 2

p <- function(theta) {
  logit_theta <- log(theta / (1 - theta))
  (1 / (sqrt(2 * pi) * sigma)) * exp(-(logit_theta - mu)^2 / (2 * sigma^2)) / (theta * (1 - theta))
}

q <- function(v) {
  dbeta(v, 1, 1)
}

k <- 5
theta_vals <- seq(0.001, 0.999, length.out = 1000)
p_vals <- p(theta_vals)
kq_vals <- k * q(theta_vals)

plot(theta_vals, p_vals, type = "l", col = "blue", lwd = 2,
     ylim = c(0, max(kq_vals)), ylab = "Density", xlab = "theta")
lines(theta_vals, kq_vals, col = "red", lwd = 2)
legend("topright", legend = c("p(theta)", "k*q(v)"), col = c("blue", "red"), lty = 1)
```



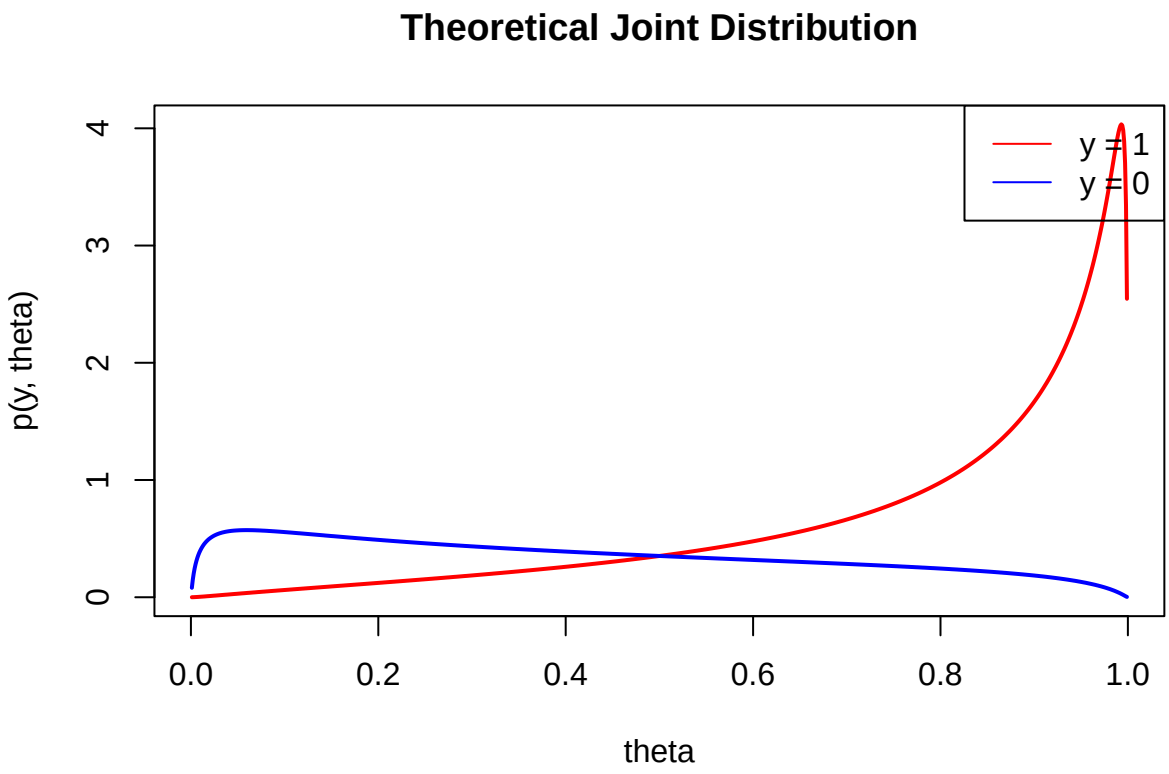

```r
rejection_sampling <- function(n) {
  samples <- c()
  while(length(samples) < n) {
    v <- rbeta(1, 1, 1)
    if (k * q(v) * runif(1) < p(v)) {
      samples <- c(samples, v)
    }
  }
  return(samples)
}

theta_samples <- rejection_sampling(10000)
```

```r
ggplot(data = data.frame(theta_samples), aes(x = theta_samples)) +
  geom_histogram(aes(y = after_stat(density)), bins = 80, fill = "blue", alpha = 0.5) +
  stat_function(fun = p, col = "red") +
  labs(title = "Rejection Sampling",
       x = "theta", y = "Density")
```

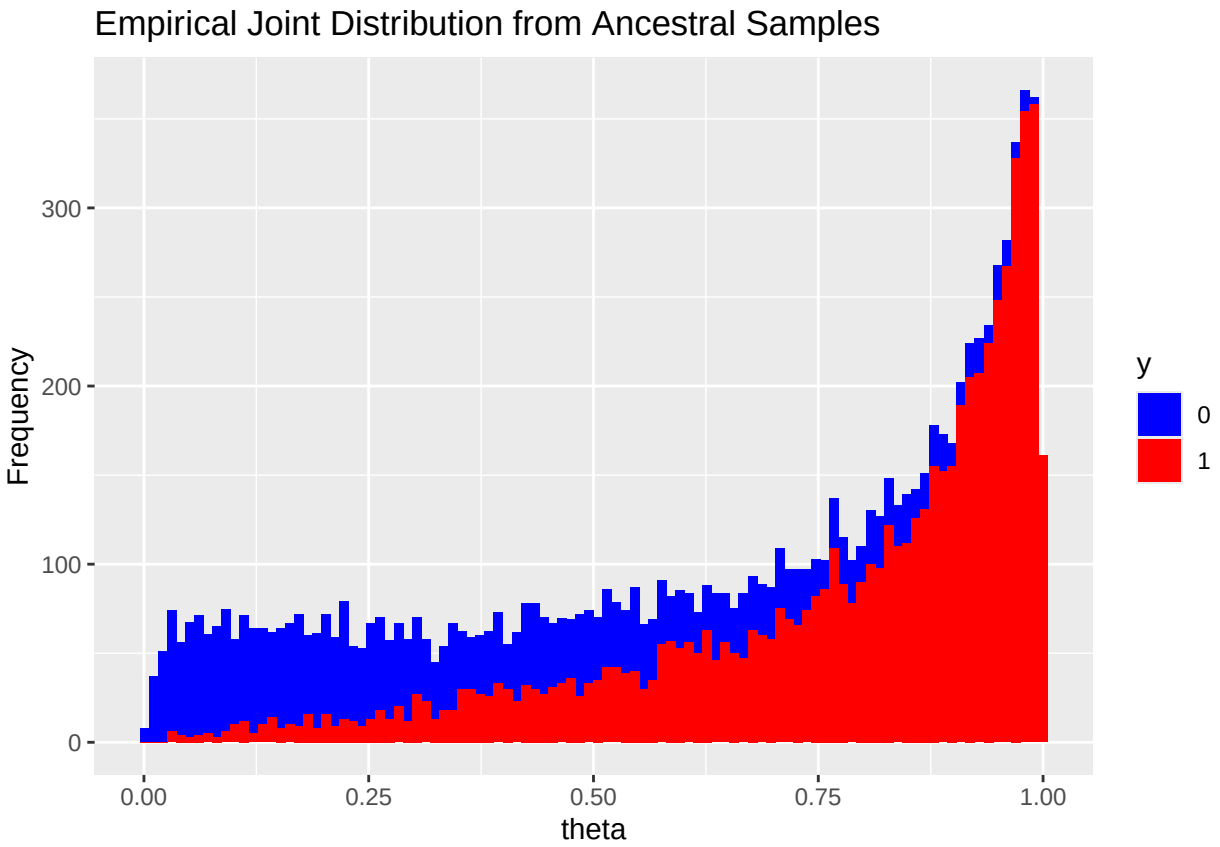


Part (b)

```r
set.seed(42)

y_samples <- rbinom(length(theta_samples), size = 1, prob = theta_samples)

p_joint_y0 <- (1 - theta_vals) * p(theta_vals)
p_joint_y1 <- theta_vals * p(theta_vals)

plot(theta_vals, p_joint_y1, type = "l", col = "red", lwd = 2,
     ylim = c(0, max(p_joint_y0, p_joint_y1)),
     xlab = "theta", ylab = "p(y, theta)",
     main = "Theoretical Joint Distribution")
lines(theta_vals, p_joint_y0, col = "blue", lwd = 2)
legend("topright", legend = c("y = 1", "y = 0"), col = c("red", "blue"), lty = 1)
```




```
joint_df <- data.frame(theta = theta_samples, y = as.factor(y_samples))

ggplot(joint_df, aes(x = theta, fill = y)) +
  geom_histogram(position = "stack", bins = 100) +
  scale_fill_manual(values = c("blue", "red")) +
  labs(title = "Empirical Joint Distribution from Ancestral Samples",
       x = "theta", y = "Frequency")
```



Observing the theoretical joint distribution plot, we see blue dominating towards theta=0, and red dominating towards theta=1. But in the histogram graph, we see blue dominating towards theta=0, and blue and red being even towards theta=1. This shows the sampler was decent, excpet for when theta is high.

Problem 3

Part (a)

```
library(mvtnorm)

p <- function(y, mu, Sigma) {
  dmvnorm(y, mean = mu, sigma = Sigma)
}

gaussian_metropolis <- function(n, mu, Sigma, sigma = 2, burn = 0.18) {
  samples <- list()
  y1 <- mu[1]
  y2 <- mu[2]
  total_samples = floor(n / (1 - burn))
  m = as.integer(total_samples * burn)
  accepted <- 0

  while(length(samples) < total_samples) {
    y1_prop <- rnorm(1, y1, sigma)
```

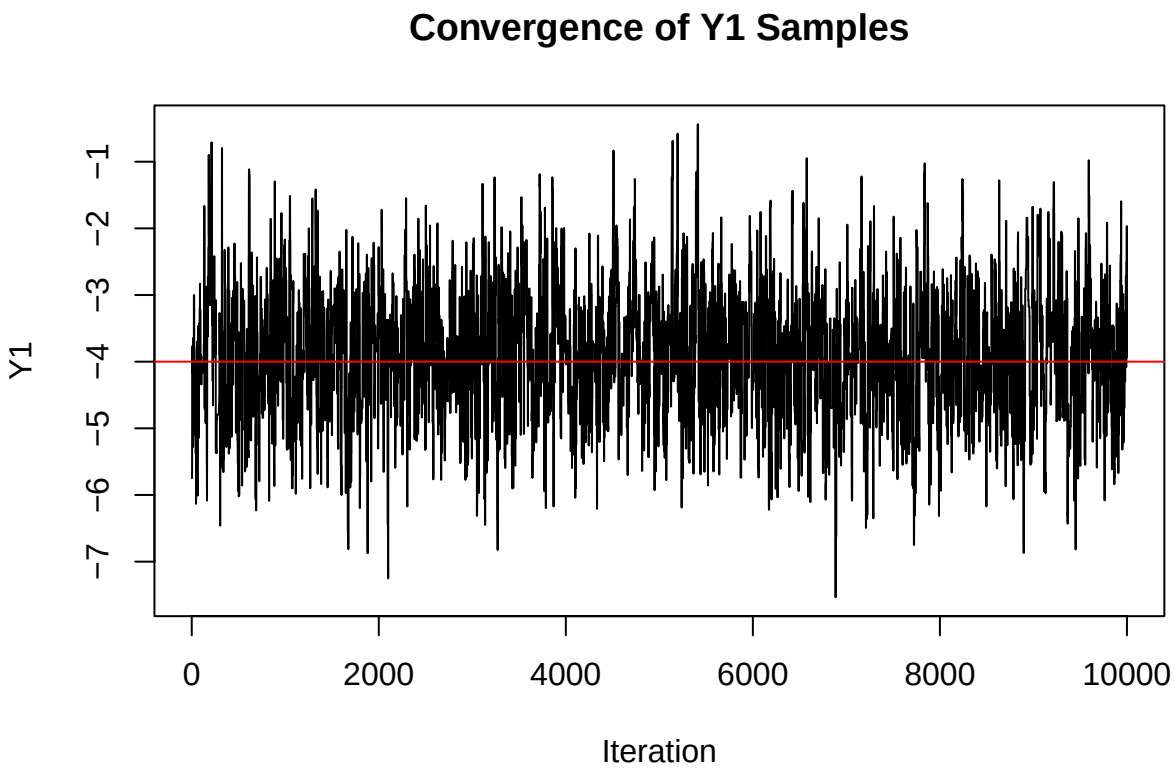
```
    y2_prop <- rnorm(1, y2, sigma)
    alpha <- p(c(y1_prop, y2_prop), mu, Sigma) / p(c(y1, y2), mu, Sigma)

    if (runif(1) < alpha) {
      y1 <- y1_prop
      y2 <- y2_prop
      accepted <- accepted + 1
    }
    samples[[length(samples) + 1]] <- c(y1, y2)
  }

  acceptance_rate <- accepted / total_samples
  cat(sprintf('Acceptance rate: %.3f\n', acceptance_rate))

  samples <- do.call(rbind, samples)
  samples <- samples[(m + 1):nrow(samples), ]

  return(samples)
}
```

Part (b)

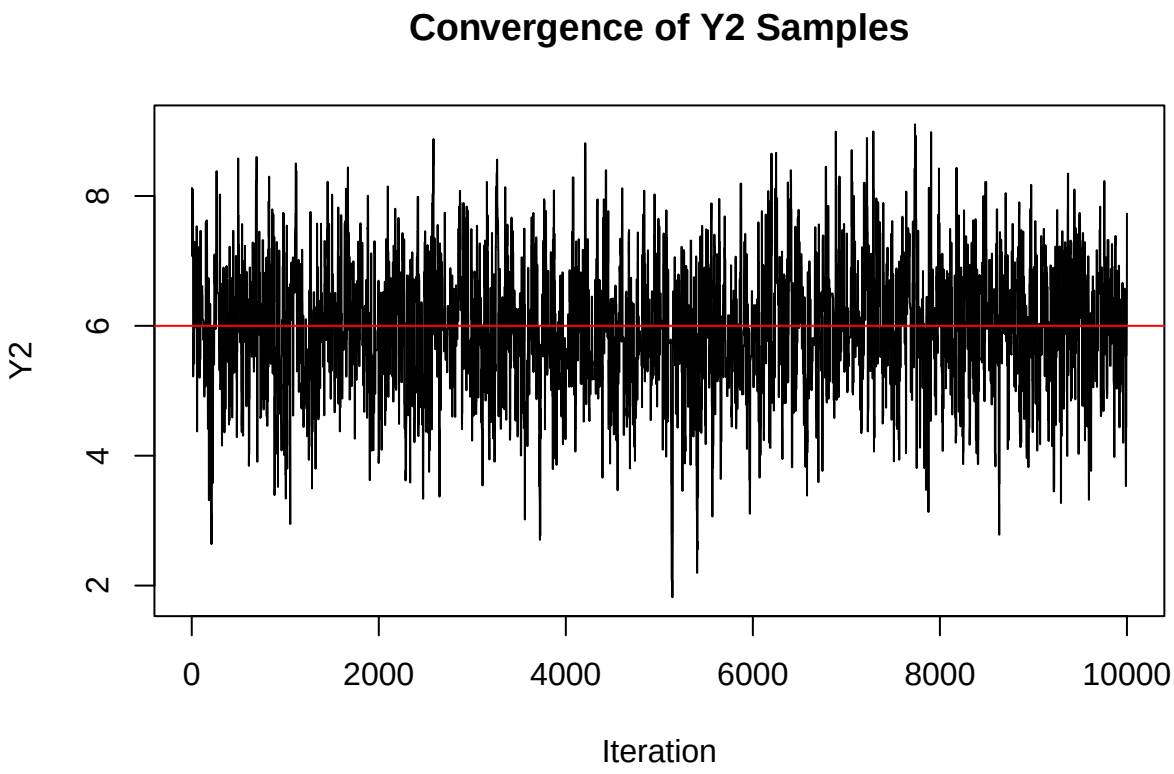
```
set.seed(42)

mu_target <- c(-4, 6)
Sigma_target <- matrix(c(1, -0.6, -0.6, 1), nrow = 2, byrow = TRUE)

samples <- gaussian_metropolis(10000, mu_target, Sigma_target, sigma = 1.5)
```

```
## Acceptance rate: 0.340
```

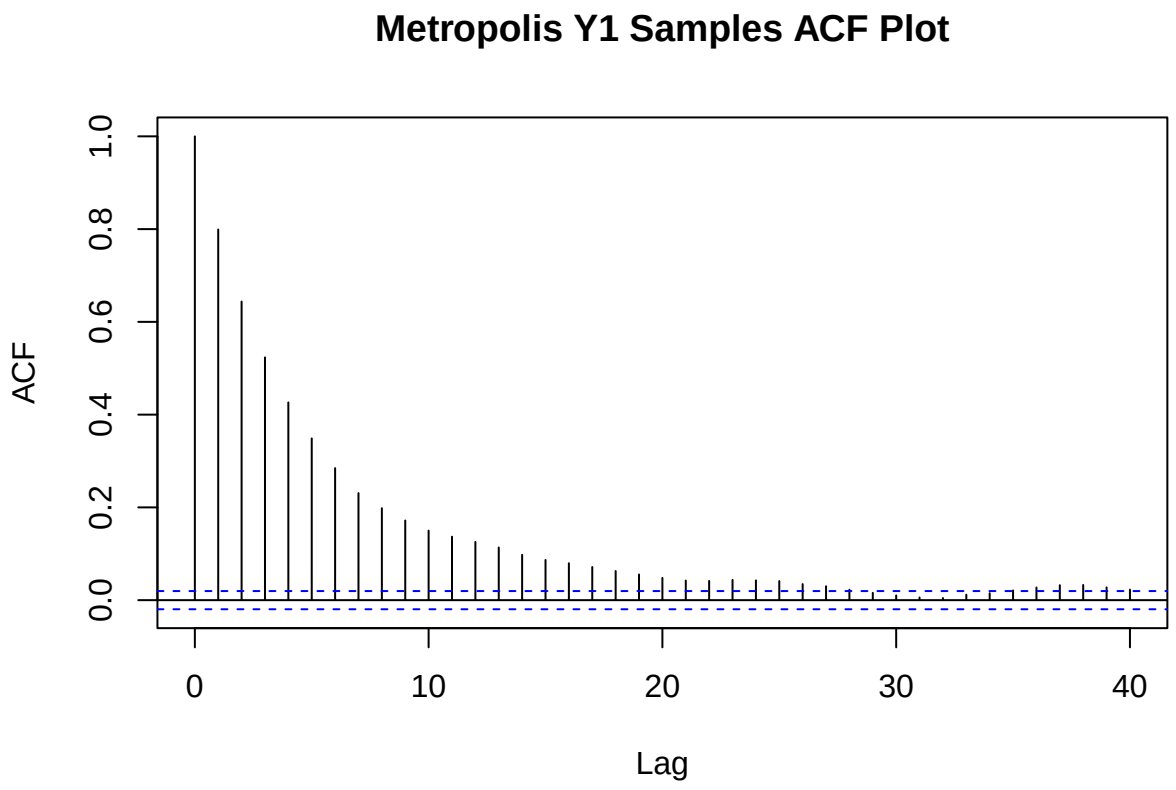
```
plot(samples[, 1], type = "l", main = "Convergence of Y1 Samples",
     xlab = "Iteration", ylab = "Y1")
abline(h = mu_target[1], col = "red", lwd = 1)
```



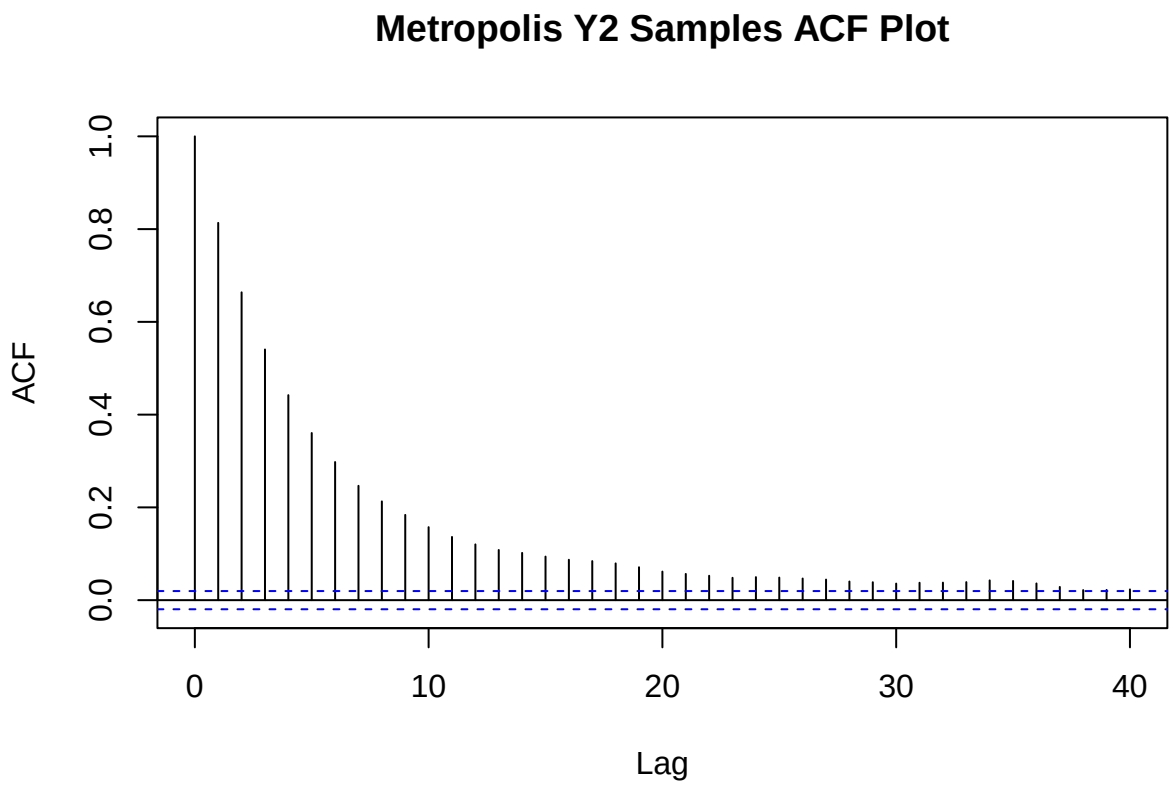

```
plot(samples[, 2], type = "l", main = "Convergence of Y2 Samples",
     xlab = "Iteration", ylab = "Y2")
abline(h = mu_target[2], col = "red", lwd = 1)
```




```r
acf(samples[, 1], main = "Metropolis Y1 Samples ACF Plot")
```




```
acf(samples[, 2], main = "Metropolis Y2 Samples ACF Plot")
```




```
print(colMeans(samples))
```

```
## [1] -3.92768  5.95716
```

```
print(cov(samples))
```

```
##            [,1]       [,2]
## [1,]  0.9402053 -0.5645958
## [2,] -0.5645958  0.9936658
```

Yes. Trace plots flucuate evenly around the mean. ACF decays smoothly down to around 0 at around lag 15. The sample mean and covariance are very close to the true mean and covariance.

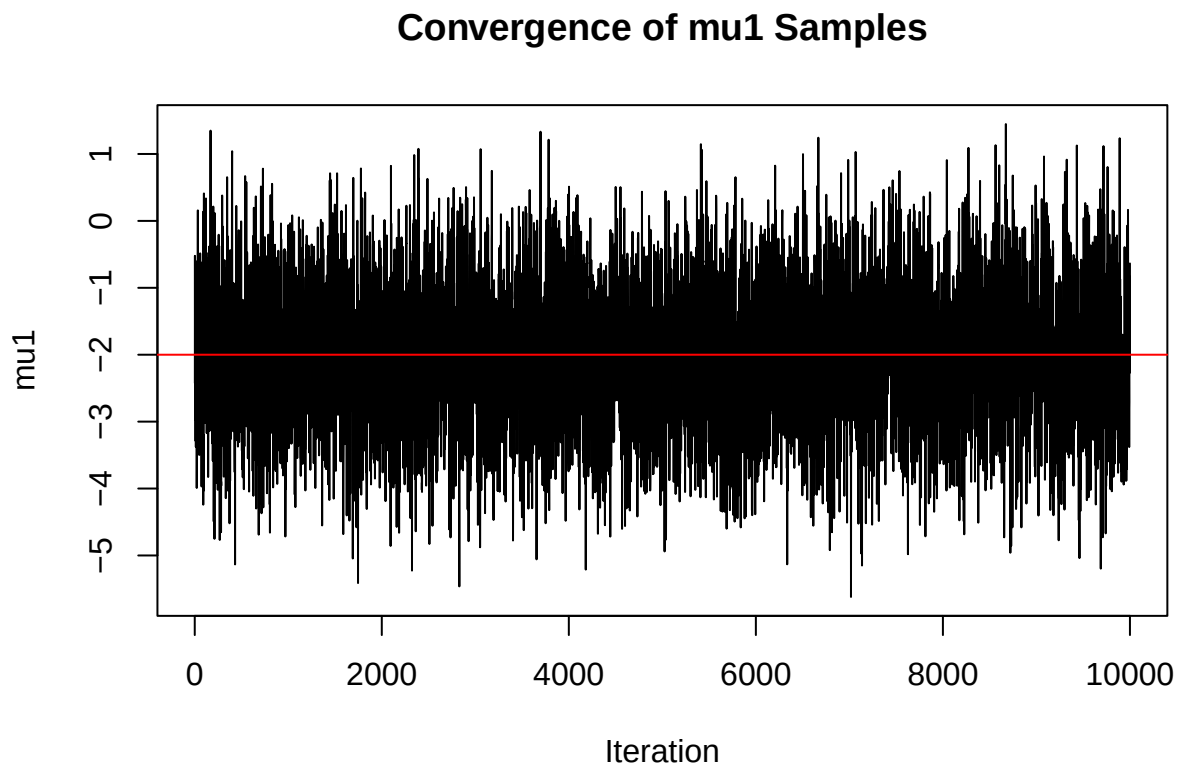
Problem 4

Part (a)

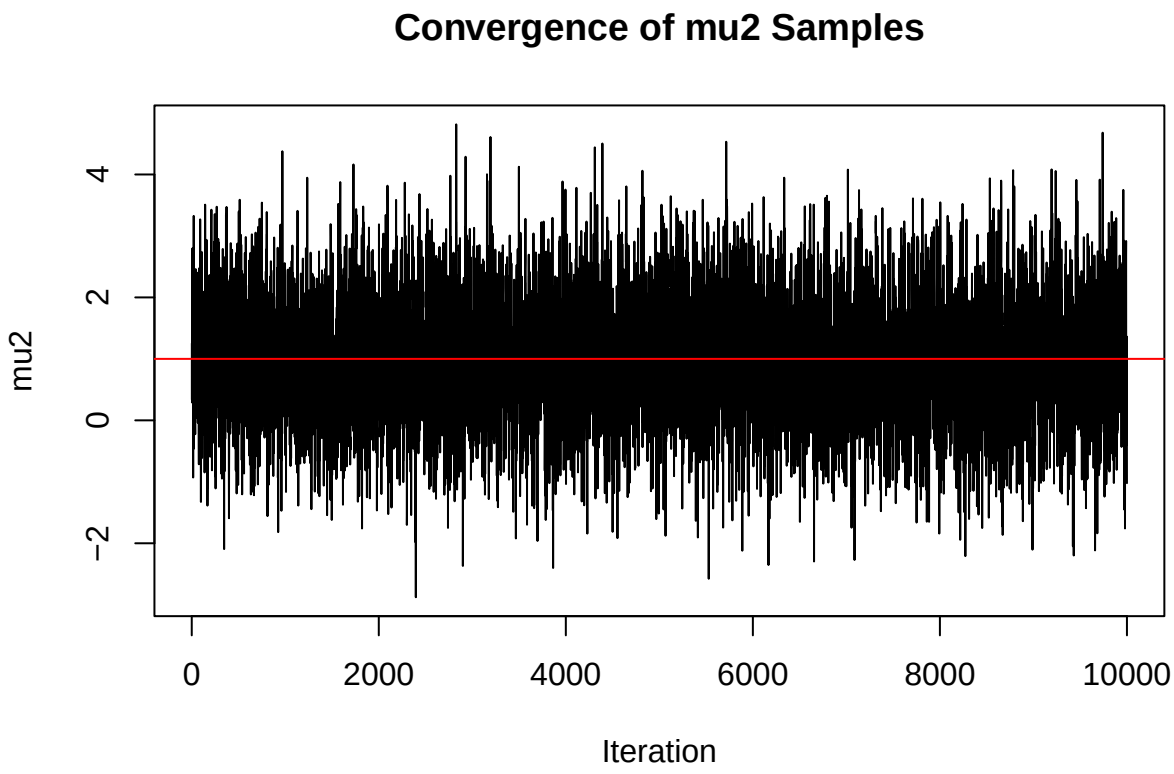
```
set.seed(42)

y_obs <- c(-2, 1)
rho <- -0.6
```

```r
gibbs_sampling <- function(n, burn = 0.18) {
  total_samples = floor(n / (1 - burn))
  m = floor(total_samples * burn)
  samples <- matrix(0, nrow = total_samples, ncol = 2)
  mu1 <- 0
  mu2 <- 0

  for(i in 1:total_samples) {
    mu1 <- rnorm(1, y_obs[1] + rho * (mu2 - y_obs[2]), sqrt(1 - rho^2))
    mu2 <- rnorm(1, y_obs[2] + rho * (mu1 - y_obs[1]), sqrt(1 - rho^2))
    samples[i, ] <- c(mu1, mu2)
  }

  return(samples[(m + 1):total_samples, ])
}

samples_gibbs <- gibbs_sampling(10000)
```

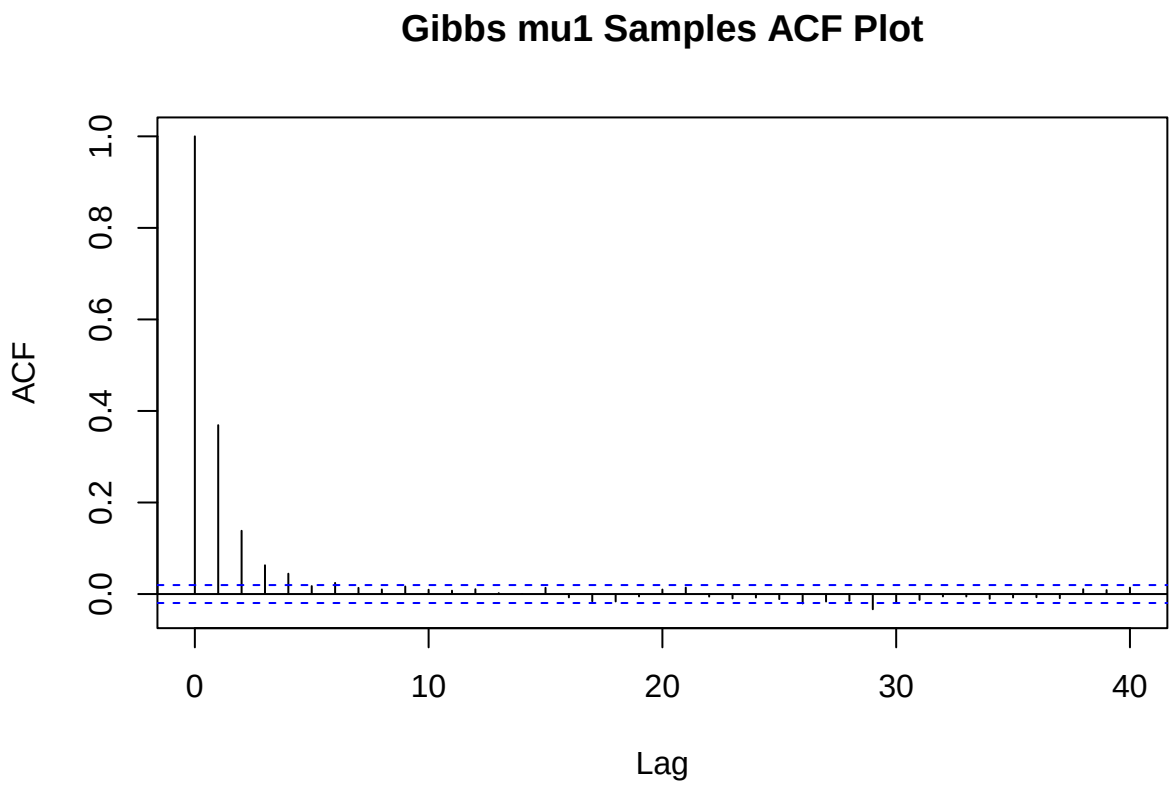
```r
plot(samples_gibbs[, 1], type = "l", main = "Convergence of mu1 Samples",
     xlab = "Iteration", ylab = "mu1")
abline(h = y_obs[1], col = "red", lwd = 1)
```



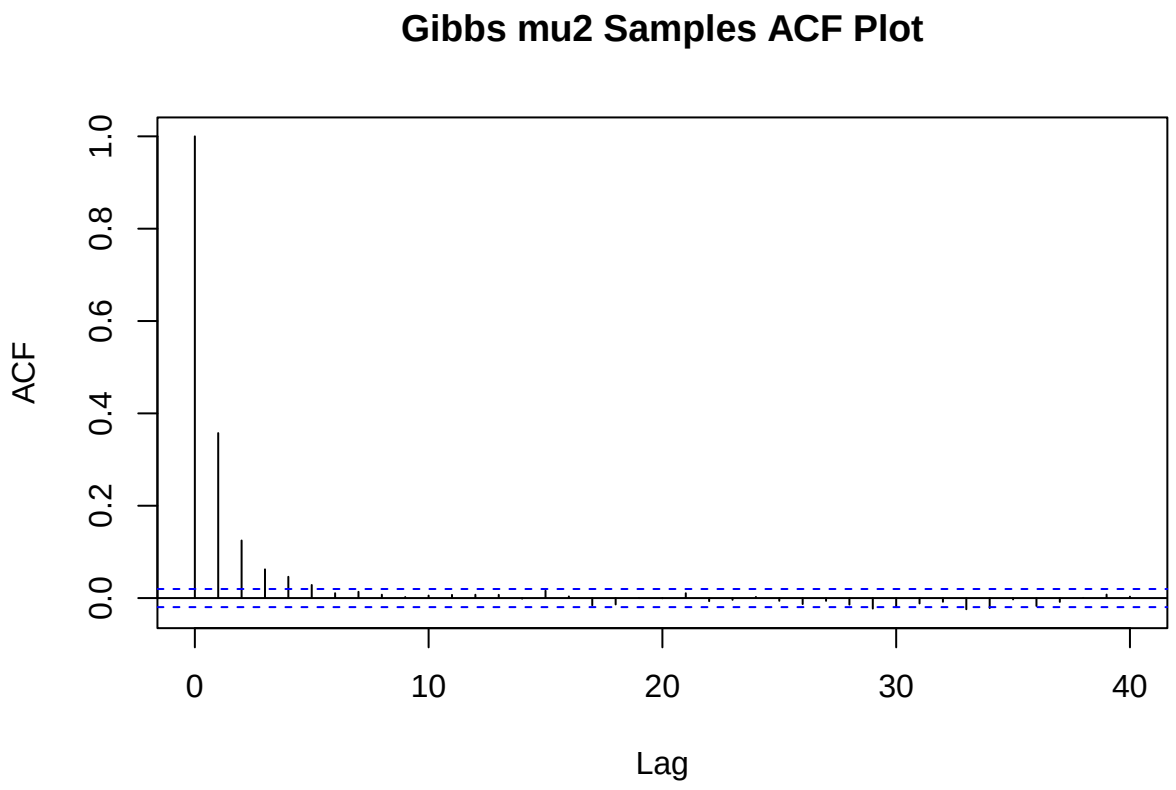

```r
plot(samples_gibbs[, 2], type = "l", main = "Convergence of mu2 Samples",
     xlab = "Iteration", ylab = "mu2")
abline(h = y_obs[2], col = "red", lwd = 1)
```




```r
acf(samples_gibbs[, 1], main = "Gibbs mu1 Samples ACF Plot")
```




```
acf(samples_gibbs[, 2], main = "Gibbs mu2 Samples ACF Plot")
```



Part (b)

```
true_mean <- y_obs
true_Sigma <- matrix(c(1, rho, rho, 1), nrow = 2, byrow = TRUE)

print(colMeans(samples_gibbs))
```

```
## [1] -2.000691  1.001081
```

```
print(true_mean)
```

```
## [1] -2  1
```

```
print(cov(samples_gibbs))
```

```
##            [,1]       [,2]
## [1,]  1.0323191 -0.6206683
## [2,] -0.6206683  1.0199546
```

```
print(true_Sigma)
```

```
##      [,1] [,2]
## [1,]  1.0 -0.6
## [2,] -0.6  1.0
```

Mean and covariance are very close to the truth. An improper prior allows for the data to fully control the posterior. But with a lack of data it can lead to meaningless results.